

Identifying Maximum Defective Bicliques in Large Bipartite Graphs

Zhiyi Wang
School of Computer Science
The University of Sydney
Sydney, Australia
zwan9517@uni.sydney.edu.au

Lijun Chang
School of Computer Science
The University of Sydney
Sydney, Australia
Lijun.Chang@sydney.edu.au

Jeffrey Xu Yu
Department of SEEM
The Chinese University of Hong Kong
Hong Kong, China
yu@se.cuhk.edu.hk

Abstract—Finding dense subgraphs in a bipartite graph is a powerful tool for uncovering meaningful patterns and extracting valuable insights across various domains. In this paper, we relax the definition of biclique to k -defective biclique by allowing up-to k missing edges, such that larger, but still dense, substructures can be identified. Then, we propose algorithms to find the defective biclique with the largest number of vertices, which is an NP-hard problem. Nevertheless, we prove that our algorithm runs in $\mathcal{O}^*(\gamma^{n+k})$ time, beating the trivial $\mathcal{O}^*(2^n)$ time complexity; here the $\mathcal{O}^*$ notation hides polynomial factors, n is the number of vertices in the input graph G and $\gamma \approx 1.8393$ is a constant. We further prove the diameter-three property of k -defective bicliques with at least $k+1$ vertices on each side, and utilize it to reduce the exponent from $n+k$ to $\alpha\Delta^2+k$ where α and Δ are the degeneracy and maximum degree of G , respectively. Finally, we propose several practical techniques (i.e., upper bounds, reduction rules, an iterative computation framework, and finding a large initial solution) to improve the practical efficiency of our algorithm. Extensive empirical studies on real bipartite graphs are conducted to evaluate our techniques. As a by-product, our analysis techniques can also be used to prove a time complexity of $\mathcal{O}^*(\gamma^{n+k})$ for maximum defective clique computation in traditional unipartite graphs, improving the state-of-the-art time complexity.

I. INTRODUCTION

Many real-world networks, such as author-paper networks [1], customer-product networks [2], and biological networks [3], are naturally represented as bipartite graphs. In a bipartite graph, its vertices are divided into two disjoint sets, each representing a distinct type of entity, and edges only exist across the two sets, representing relationships or interactions between entities in the two sets. Cohesive (i.e., dense) subgraph discovery in bipartite graphs has become a powerful tool for uncovering meaningful patterns and extracting valuable insights across various domains. For instance, cohesive subgraphs in interaction-type bipartite graphs such as author-conference networks can highlight research communities [4], [5], [6]; that is, researchers frequently publishing at the same venues form dense subgraphs. Cohesive subgraphs in user-item bipartite graphs play a significant role in improving recommendation accuracy [7], [8]. In gene expression networks, genes often exhibit correlations across specific sets of samples, forming local patterns that manifest as dense subgraphs; detecting them can aid in the identification of functional relationships and underlying biological processes [9], [10].

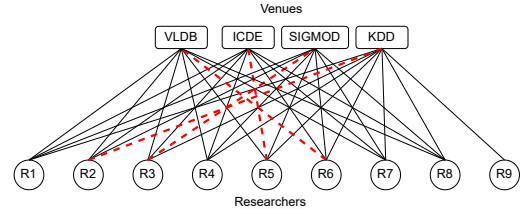


Fig. 1: An example bipartite graph of researchers-publishing-in-venues (red dashed lines are missing edges)

The classical biclique model, requiring all vertices in one side (i.e., set) to be fully connected to all vertices on the other side, is the densest substructure that can exist in a bipartite graph. For example, for the bipartite graph in Figure 1 that represents researchers publishing in conference venues, researchers $\{R1, R4, R7, R8\}$ together with conference venues $\{VLDB, ICDE, SIGMOD, KDD\}$ form a maximal biclique. Both the problem of enumerating all maximal bicliques and the problem of finding the largest biclique have been extensively studied recently in view of their practical importance, e.g., see [11], [12] and references therein. Nevertheless, the biclique model could be overly restrictive for many applications. Firstly, the data collection process may introduce errors and be incomplete [13]. Secondly, missing a few edges from being a biclique still preserves most of its overall cohesion. In view of this, we extend the concept of “defectiveness” from cliques in unipartite (i.e., traditional) graphs [13], [14] to bicliques in bipartite graphs. Specifically, a k -defective biclique allows up-to k missing edges from being a biclique, offering a more flexible approach for identifying cohesive substructures in bipartite graphs. The entire graph in Figure 1 after excluding R9 is a 4-defective biclique. Defective bicliques are useful in scenarios where relationships between entities exhibit dense but incomplete connections. Besides the applications mentioned above, another important application area is in protein interaction networks, where protein-protein interactions often form near-complete bipartite relationships, but not all protein pairs have direct interactions [15]. Defective biclique allows for missing edges, enabling the identification of functionally cohesive protein groups despite incomplete interaction data. This helps prevent the omission of crucial structures and facilitates the discovery of latent biological interactions.

In this paper, we study the maximum defective biclique computation problem, which aims to find the k -defective biclique with the largest number of vertices and at least τ vertices on each side for a user given threshold τ that is no smaller than $k+1$. This problem is NP-hard. A naive approach that exhaustively enumerates all possible vertex subsets results in a time complexity of $\mathcal{O}^*(2^n)$; here, the $\mathcal{O}^*$ notation hides polynomial factors. To achieve a sub- $\mathcal{O}^*(2^n)$ time complexity, we develop a backtracking algorithm with two simple reduction rules, and prove that it runs in $\mathcal{O}^*(\gamma^{n+k})$ time, where $\gamma \approx 1.8393$ is the real root of the equation $x^3 - x^2 - x - 1 = 0$. Our time complexity analysis technique is different from that of [16], [17], [18] for maximum k -defective clique computation in unipartite graphs, by noticing that the time complexity for the latter is $\mathcal{O}^*(\gamma_k^n)$ in [17] where γ_k depends on k and is the largest real root of the equation $x^{k+3} - 2x^{k+2} + 1 = 0$; $\gamma_1 = \gamma$ and γ_k quickly approaches 2 when k increases. We remark that, by applying our analysis technique, we can also prove a time complexity of $\mathcal{O}^*(\gamma^{n+k})$ for the algorithm of [17] for maximum defective clique computation. Furthermore, we prove the diameter-three property of large defective bicliques: any k -defective biclique with at least $k+1$ vertices on each side has a diameter at most 3. By utilizing this property, we further improve our time complexity to $\mathcal{O}^*(\gamma^{\alpha\Delta^2+k})$, where α and Δ are the degeneracy and maximum degree of the input graph, respectively. The general idea is that, once a vertex has been added to the partial (*i.e.*, growing) solution during backtracking, all vertices with distances larger than 3 to this newly added vertex can be removed from consideration, significantly reducing the search space and time complexity.

We further incorporate several practical techniques to improve the practical efficiency of our algorithm. Firstly, we propose two upper bounds which are used to prune a backtracking search instance when either the computed total upper bound is no larger than the size of the currently found best solution or single-side upper bound is smaller than τ , and additional reduction rules to shrink a backtracking instance by removing unpromising candidate vertices. Secondly, to enhance the effectiveness of pruning techniques that exploit the user-given threshold τ as a single-side lower bound for pruning, we incorporate the progressive bounding idea of [12] to manually set larger single-side lower bounds by enumerating possible size ranges of the two sides of the optimal solution. Thirdly, we observe that (1) the search space increases dramatically as k grows, while (2) the maximum k -defective biclique with at least τ vertices on each side exceeds the size of the maximum $(k-1)$ -defective biclique with the same property (if exists) by at most 1. In view of this, we propose an iterative computation framework that iteratively computes the maximum s -defective biclique with at least τ vertices on each side, for s ranging from 0 to k . When searching for the maximum s -defective biclique, we have both a tight lower bound (which is the size of the maximum $(s-1)$ -defective biclique) and a tight upper bound (which is the lower bound plus one) of the optimal size, enabling efficient search. Finally, we also propose techniques to efficiently find a large initial k -defective biclique.

Contributions. Our main contributions are as follows:

- We introduce the problem of finding the maximum k -defective biclique and develop general frameworks to solve it in $\mathcal{O}^*(\gamma^{n+k})$ and $\mathcal{O}^*(\gamma^{\alpha\Delta^2+k})$ time; here, $\gamma \approx 1.8393$ is the largest real root of the equation $x^3 - x^2 - x - 1 = 0$, and α and Δ are the degeneracy and maximum degree of the input graph, respectively.
- We propose several practical upper bounds and reduction rules to improve the practical efficiency of our algorithm.
- We develop an iterative computation framework that computes the maximum s -defective biclique by iteratively increasing s starting from 0. A notable feature of this approach is that it allows us to avoid specifying k in the input and instead stop the algorithm as needed.
- We propose efficient and effective techniques for finding a large initial (*i.e.*, heuristic) k -defective biclique.

We also conduct extensive empirical studies on large real bipartite graphs to evaluate our techniques. The results show that (1) solving the maximum k -defective biclique problem enables us to find larger, but still dense, substructures than bicliques in bipartite graphs, and (2) each of our practical techniques contributed to the practical efficiency of our algorithm. Omitted proofs can be found in the full version.¹

II. RELATED WORK

Cohesive subgraphs in bipartite graphs. In recent years, there has been significant research on identifying cohesive subgraphs in bipartite graphs, including (α, β) -core [19], k -bitruss [20], biclique [12], k -biplex [21], and quasi-biclique [22]. An (α, β) -core is a maximal bipartite subgraph where each vertex on one side is connected to at least α vertices on the other side, and vice versa. A k -bitruss is a bipartite subgraph where each edge is part of at least k butterflies (*i.e.*, 2×2 bicliques). Subgraphs identified by these two models are usually large and sparse, as the parameters α , β , and k could be much smaller than the number of vertices in a result. Biclique is the densest substructure that can exist in a bipartite graph. Enumerating all maximal bicliques has been studied in [11], [23], [24], [25], [26], counting and enumerating bicliques of a specific size is studied in [27], finding a small number of large bicliques is studied in [12], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37]. k -biplex relaxes biclique by allowing each vertex to be disconnected from at most k vertices on the opposite side. Enumerating all maximal biplexes has been studied in [21], [38], while maximum biplex search has been studied in [39], [40]. Quasi-biclique is a variant of biplex that allows each vertex to be disconnected from a proportion of vertices on the opposite side, controlled by a parameter $\theta \in [0, 1)$. While quasi-biclique offers a more flexible structure, existing methods rely on mixed integer programming to compute the maximum quasi-biclique exactly [22], which is extremely slow for large graphs. None of these algorithms can be used for finding the maximum defective biclique due to different problem definitions.

¹<https://github.com/zhiyiwang1115/defectiveBiclique>

Maximum defective clique computation over unipartite graphs. The concept of k -defective clique, which allows up-to k missing edges from a clique, in a unipartite (*i.e.*, traditional) graph was first introduced in [14]. Finding a k -defective clique with the largest number of vertices, although is an NP-hard problem, has been extensively studied recently. An algorithm with time complexity $\mathcal{O}^*(\gamma_{2k}^n)$ was proposed in [16], where γ_i is the largest real root of the equation $x^{i+3} - 2x^{i+2} + 1 = 0$. Later, the time complexity was improved to $\mathcal{O}^*(\gamma_k^n)$ in [17] and $\mathcal{O}^*(\gamma_{k-1}^n)$ in [18]. Designing practical algorithms without considering time complexity was also studied in [41], [42]. These techniques cannot be directly applied to bipartite graphs, and also extending them to bipartite graphs may not lead to practical performance improvement. Notably, the most powerful reduction rules for defective clique computation are based on graph coloring [17]. To extend it to bipartite graphs, we would need to (virtually) convert a bipartite graph to a unipartite graph by connecting all vertices on the same side to each other; this is because vertices from the same side does not introduce non-edges. Then, all vertices from the same side receive distinct colors, which eliminates the potential for pruning. In this paper, we introduce progressive bounding and iterative computation to achieve significant performance improvements. Moreover, as discussed at the end of Section IV-A, our analysis techniques can also be used to prove a better time complexity for maximum defective clique computation.

III. PRELIMINARIES

In this paper, we consider an *undirected and unweighted bipartite graph* $G = (U, V, E)$, where U and V are two disjoint vertex sets (referred to as the two *sides* of the bipartite graph) and $E \subseteq U \times V$ is an edge set; $(U \times V) \setminus E$ is referred to as the set of *missing edges* of G . We use u (resp. v) to refer to vertices of U (resp. V), and use w to refer to vertices of $U \cup V$ (*i.e.*, from either U or V). The number of vertices and the number of edges in G are denoted by $n = |U| + |V|$ and $m = |E|$, respectively. For a vertex $u \in U$, we denote the set of neighbors of u in V by $N_u(V) = \{v \in V \mid (u, v) \in E\}$, the degree of u in V by $d_u(V) = |N_u(V)|$, and the *non-degree* of u in V by $\bar{d}_u(V) = |V| - d_u(V)$; $V \setminus N_u(V)$ is referred to as the set of *non-neighbors* of u . The neighbors, degrees, non-neighbors and non-degrees of vertices of V are defined similarly. Note that, *two vertices from the same side are considered neither neighbor nor non-neighbor of each other*. Thus, $d_u(V \cup U) = d_u(V)$ and $\bar{d}_u(V \cup U) = \bar{d}_u(V)$. Given a pair of vertex sets (S_U, S_V) where $S_U \subseteq U$ and $S_V \subseteq V$, the subgraph of G induced by (S_U, S_V) is denoted by $G[S_U, S_V] = (S_U, S_V, \{(u, v) \in E \mid u \in S_U, v \in S_V\})$.

Definition 3.1: (Biclique) A bipartite graph $G = (U, V, E)$ is a biclique if $E = U \times V$.

Definition 3.2: (k -Defective Biclique) A bipartite graph $G = (U, V, E)$ is a k -defective biclique if $|(U \times V) \setminus E| \leq k$ or equivalently, $|E| \geq |U| \cdot |V| - k$.

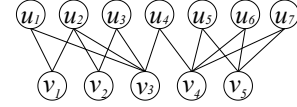


Fig. 2: An example bipartite graph

The definition of k -defective biclique relaxes the definition of biclique by allowing up-to k missing edges, and a biclique is a special case of k -defective biclique for $k = 0$. It is easy to see that any *maximal* k -defective biclique is a vertex-induced subgraph, and the k -defective biclique property is hereditary (*i.e.*, any vertex subset of a k -defective biclique is also a k -defective biclique). Thus, in the remainder of the paper, we mostly consider vertex-induced subgraphs, and refer to a vertex-induced subgraph $G[R_U, R_V]$ by its vertex set (R_U, R_V) .

Example 3.1: Consider the graph in Figure 2. The vertex subset $(\{u_5, u_6, u_7\}, \{v_4, v_5\})$ forms a maximal biclique. Extending this, $(\{u_4, u_5, u_6, u_7\}, \{v_4, v_5\})$ forms a maximal 1-defective biclique with the edge (u_4, v_5) missing. Furthermore, $(\{u_1, u_2, u_3\}, \{v_1, v_2, v_3\})$ forms a maximal 2-defective biclique with the edges (u_1, v_2) and (u_3, v_1) missing. $\square$

Problem Statement. In this paper, we measure the size of a k -defective biclique (R_U, R_V) by its number of vertices (*i.e.*, $|R_U| + |R_V|$), and aim to find the k -defective biclique with the largest number of vertices. However, k -defective bicliques with an exceptionally small number of vertices on one *side* (*i.e.*, either R_U or R_V) often lack practical utility. Hence, it is more insightful to focus on k -defective bicliques that exhibit substantial sizes on both sides.

Problem (Maximum Defective Biclique Computation). Given a bipartite graph G and two positive integers k and τ satisfying $\tau \geq k + 1$, our problem is to find the maximum k -defective biclique in G with both sides having at least τ vertices.²

We will prove later in Lemma 4.2 that such k -defective bicliques (*i.e.*, with at least $k + 1$ vertices on each side) have their diameters bounded by 3. Note that, on the other hand, the only diameter-two substructures in a bipartite graph are bicliques. Thus, such k -defective bicliques are densely connected, making it a more probable substructure for dense communities in bipartite graphs.

Frequently used notations are summarized in Table I. For notation simplicity, for a vertex set S and a vertex u , we will write $S \cup u$ and $S \setminus u$ for $S \cup \{u\}$ and $S \setminus \{u\}$, respectively.

IV. OUR APPROACH WITH NONTRIVIAL TIME COMPLEXITIES

The special case of the maximum k -defective biclique computation problem for $k = 0$ is NP-hard, which can be shown by reducing from the maximum balanced biclique problem [34]. Nevertheless, we in this section propose techniques to find

²Note that, we set the same size threshold τ for both sides for simplicity, while our techniques can be straightforwardly extended to handle different size thresholds on the two sides. Also note that, although we focus on finding one largest defective biclique in this paper, our techniques can be easily extended to find multiple large defective bicliques by using the same idea as [12].

TABLE I: Frequently used notations

Notation	Meaning
$G = (U, V, E)$	a bipartite graph with vertex sets U, V and edge set E
n, m	$n = U + V $, $m = E $
(R_U, R_V)	vertex set or subgraph induced by $R_U \subseteq U$ and $R_V \subseteq V$
$N_u(S)$	the set of u 's neighbors that are in S
$d_u(S)$	the number of u 's neighbors that are in S
$\bar{d}_u(S)$	the number of u 's non-neighbors that are in S
α	degeneracy of G by treating it as a unipartite graph
Δ	$\Delta = \max\{\max_{u \in U} d_u(V), \max_{v \in V} d_v(U)\}$
I	a backtracking instance, $I = (g, R_U, R_V, C_U, C_V)$
$\bar{e}(I)$	the number of missing edges in $G[I, R_U, I, R_V]$

the maximum defective biclique in time faster than the trivial $\mathcal{O}^*(2^n)$ time complexity; here, the $\mathcal{O}^*$ notation omits polynomial factors. Specifically, we first show in Section IV-A that the maximum defective biclique can be found in $\mathcal{O}^*(\gamma^{n+k})$ time, where $\gamma \approx 1.8393$ is the real root of the equation $x^3 - x^2 - x - 1 = 0$. We then further improve the time complexity to $\mathcal{O}^*(\gamma^{\alpha\Delta^2+k})$ in Section IV-B, where α is the degeneracy and Δ is the maximum degree of the input graph G . Note that, we mainly focus on techniques for achieving our theoretical time complexities in this section, while practical techniques will be introduced in Section V.

A. Achieve $\mathcal{O}^*(\gamma^{n+k})$ Time Complexity

As the problem is NP-hard, we resort to the general paradigm of branch-and-bound (aka, backtracking) search which is widely used for finding the optimal solution to NP-hard problems. The general methodology can be described as follows. Let (g, R_U, R_V, C_U, C_V) represent an instance within the backtracking search. Here, (R_U, R_V) stores the *growing partial solution*, while C_U and C_V are sets of *candidate vertices* that are used for growing R_U and R_V , respectively. The graph g is a subgraph of the input graph G that includes vertices $R_U \cup R_V \cup C_U \cup C_V$ and (possibly only a subset of) the edges connecting these vertices in G ; in most scenarios, g is simply the subgraph of G induced by $(R_U \cup C_U, R_V \cup C_V)$. Additionally, there are globally available constants k and τ . The objective of solving an instance (g, R_U, R_V, C_U, C_V) is to identify the largest k -defective biclique within the instance, where a k -defective biclique is said to be within the instance if it includes all vertices of $R_U \cup R_V$ and no vertices outside $R_U \cup R_V \cup C_U \cup C_V$, and has at least τ vertices on each side.

To solve the instance (g, R_U, R_V, C_U, C_V) , a backtracking algorithm selects a branching vertex $w \in C_U \cup C_V$, and then recursively resolves two new instances that are generated based on the branching vertex w . One instance adds w into R_U or R_V (depending on whether w is from C_U or C_V), while the other instance removes w from $C_U \cup C_V$ (and also from g). Solving the instance $(G, \emptyset, \emptyset, U, V)$ thus identifies the maximum k -defective biclique in G . All instances generated during the resolution of $(G, \emptyset, \emptyset, U, V)$ form a binary search tree (see Figure 3), where each node represents a backtracking instance and has two children representing the two new instances generated based on a selected branching vertex.

Our algorithm uses the following simple branching rule.

Branching Rule (BR). Given an instance (g, R_U, R_V, C_U, C_V) , if there exists a vertex $w \in C_U \cup C_V$ that has at least

Algorithm 1: Branch&Bound(g, R_U, R_V, C_U, C_V)

```

1  $(g', R'_U, R'_V, C'_U, C'_V) \leftarrow$  apply reduction rules RR1 and RR2
  to  $(g, R_U, R_V, C_U, C_V)$ ;
2 if  $C'_U \cup C'_V = \emptyset$  and  $|R'_U| \geq \tau$  and  $|R'_V| \geq \tau$  then
3   Update  $(R_U^*, R_V^*)$  by  $(R'_U, R'_V)$ ; /*  $(R_U^*, R_V^*)$  is a
   global variable used to store the
   current best solution */;
4 if there exists a vertex of  $C'_U \cup C'_V$  that has at least one
   non-neighbor in  $R'_U \cup R'_V$  then
5    $w \leftarrow$  a vertex of  $C'_U \cup C'_V$  that has at least one
   non-neighbor in  $R'_U \cup R'_V$ ;
6 else  $w \leftarrow$  any vertex of  $C'_U \cup C'_V$ ;
7 if  $w \in C'_U$  then
8   Branch&Bound( $g', R'_U \cup w, R'_V, C'_U \setminus w, C'_V$ );
9   Branch&Bound( $g', R'_U, R'_V, C'_U \setminus w, C'_V$ );
10 else
11   Branch&Bound( $g', R'_U, R'_V \cup w, C'_U, C'_V \setminus w$ );
12   Branch&Bound( $g', R'_U, R'_V, C'_U, C'_V \setminus w$ );

```

one non-neighbor in $R_U \cup R_V$, then such a w is chosen as the branching vertex. Otherwise, an arbitrary vertex from $C_U \cup C_V$ is chosen as the branching vertex.

To achieve our desired time complexity, we also incorporate the following two reduction rules into our backtracking search.

Reduction Rule (RR1). Given an instance (g, R_U, R_V, C_U, C_V) , for a vertex $u \in C_U$ (resp. $v \in C_V$), if $(R_U \cup u, R_V)$ (resp. $(R_U, R_V \cup v)$) is not a k -defective biclique, then we can remove u (resp. v) from C_U (resp. C_V) and g .

Reduction Rule (RR2). Given an instance (g, R_U, R_V, C_U, C_V) , for a vertex $u \in C_U$ (resp. $v \in C_V$) that is adjacent to all vertices of $R_V \cup C_V$ (resp. $R_U \cup C_U$), we can greedily add u (resp. v) to R_U (resp. R_V).

The correctness of reduction rule **RR1** directly follows from the hereditary property of k -defective biclique, while the correctness of reduction rule **RR2** is trivial.

Based on the above discussions, the pseudocode of our backtracking search, denoted Branch&Bound, is presented in Algorithm 1. (R_U^*, R_V^*) is a global variable that is initialized as $(\emptyset, \emptyset)$ and is used to store the currently found best solution. After invoking Branch&Bound($G, \emptyset, \emptyset, U, V$), (R_U^*, R_V^*) stores the best solution, i.e., maximum defective biclique in G . In Branch&Bound, we first apply reduction rules **RR1** and **RR2** to reduce the instance (g, R_U, R_V, C_U, C_V) to a potentially smaller instance $(g', R'_U, R'_V, C'_U, C'_V)$ satisfying $R_U \subseteq R'_U$, $R_V \subseteq R'_V$, $C'_U \subseteq C_U$, and $C'_V \subseteq C_V$ (Line 1). If both C'_U and C'_V are empty and $\min\{|R'_U|, |R'_V|\} \geq \tau$ (Line 2), we update (R_U^*, R_V^*) by (R'_U, R'_V) (Line 3); that is, if $|R'_U| + |R'_V| > |R_U| + |R_V|$, then (R_U^*, R_V^*) is updated as (R'_U, R'_V) . If $C'_U \cup C'_V$ is non-empty, we select a branching vertex w based on the branching rule **BR** (Lines 4–6) and then generate two new instances of Branch&Bound based on w (Lines 7–12). Note that, in Algorithm 1 we do not explicitly check whether (R'_U, R'_V) is a valid k -defective biclique or not; this is not necessary due to our application of **RR1**.

w_i is the vertex with the smallest degree in the subgraph of G induced by $\{w_i, \dots, w_n\}$. Then, for each vertex w_i (Line 3), we compute the largest diameter-three k -defective biclique in which the first vertex, according to the degeneracy ordering, is w_i (Lines 4–8). We achieve this by first obtaining the set C of vertices whose shortest distances, computed in the subgraph of G induced by $\{w_i, \dots, w_n\}$, to w_i are at most 3 (Line 4), and then letting $C_U = C \cap U$ and $C_V = C \cap V$ (Line 5). Note that, when invoking Branch&Bound for this C_U and C_V , we move w_i from $C_U \cup C_V$ to $R_U \cup R_V$ (Lines 6–8).

Example 4.2: Reconsider Example 4.1. One possible degeneracy ordering is $\{u_7, v_5, u_6, u_5, v_4, u_4, u_1, v_1, u_2, v_2, u_3, v_3\}$. When iterating through the vertices according to the degeneracy ordering, we only need to consider subgraphs with diameter at most three given that $\tau \geq k + 1$. For the first vertex u_7 , the MDBC algorithm is invoked with $(R_U, R_V, C_U, C_V) = (\{u_7\}, \emptyset, \{u_4, u_5, u_6\}, \{v_3, v_4, v_5\})$ where u_1, u_2, u_3, v_1, v_2 are pruned. Similarly, for v_4 , MDBC is invoked with input $(\emptyset, \{v_4\}, \{u_1, u_2, u_3, u_4\}, \{v_3\})$ where $u_5, u_6, v_7, v_1, v_2, v_5$ are pruned. We can see that in each of the n invocations to MDBC, the input is a small subgraph rather than the entire graph, improving the efficiency. $\square$

Theorem 4.2: Let P_1 be the time complexity of running Lines 1–6 of Algorithm 1, which is polynomial regarding n . The time complexity of MDBC is $\mathcal{O}(n \times P_1 \times \gamma^{\alpha \Delta^2 + k})$ and is $\mathcal{O}^*(\gamma^{\alpha \Delta^2 + k})$, where α is the degeneracy of G and Δ is the maximum degree of G .

V. PRACTICAL TECHNIQUES

Algorithm 2 only illustrates the essential techniques that are required to achieve the desired time complexity. We in this section propose techniques to improve its practical performance. Specifically, we first introduce practical pruning techniques in Section V-A. Then, we adopt a progressive bounding framework to tighten the effective size threshold τ in Section V-B, and propose an iterative computation framework by exploiting the structural properties of the k -defective biclique for different k values in Section V-C. Finally, heuristic algorithms for initially computing a large k -defective biclique are detailed in Section V-D.

A. Practical Pruning Techniques

We in this subsection introduce practical pruning techniques (i.e., upper bounds and additional reduction rules) to improve the practical performance of Branch&Bound. Specifically, the additional reduction rules will be applied at Line 1 of Algorithm 1 concurrently with **RR1** and **RR2**. The upper bounds will be checked immediately before Line 4 of Algorithm 1 so that the entire instance is pruned if either the computed total upper bound is no larger than the size of the currently found best solution (i.e., $|R_U^*| + |R_V^*|$) or a single-side upper bound is smaller than τ .

We first consider upper bounds from the aspect of $C_U \cup C_V$.

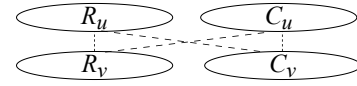


Fig. 4: Proof of **UB1** (dashed lines indicate missing edges)

Upper Bound (UB1). Given an instance $I = (g, R_U, R_V, C_U, C_V)$, let $(u_1, u_2, \dots)$ be an ordering of C_U in non-decreasing order regarding their non-degrees in $R_U \cup R_V$, and $(v_1, v_2, \dots)$ be such an ordering of C_V . Let $(w_1, w_2, \dots)$ be an ordering of $\{u_{\max\{0, \tau - |R_U| + 1\}}, u_{\max\{0, \tau - |R_U| + 2\}}, \dots\} \cup \{v_{\max\{0, \tau - |R_V| + 1\}}, v_{\max\{0, \tau - |R_V| + 2\}}, \dots\}$ in non-decreasing order regarding their non-degrees in $R_U \cup R_V$. Then, any k -defective biclique in this instance contains

- 1) at most $|R_U| + j$ vertices on the U -side, where j is the largest index such that $\sum_{i=1}^j \bar{d}_{u_i}(R_V) \leq k - \bar{e}(I) - \sum_{i=1}^{\tau - |R_U|} \bar{d}_{v_i}(R_U)$ (this is single-side upper bound).
- 2) at most $|R_V| + j$ vertices on the V -side, where j is the largest index such that $\sum_{i=1}^j \bar{d}_{v_i}(R_U) \leq k - \bar{e}(I) - \sum_{i=1}^{\tau - |R_U|} \bar{d}_{u_i}(R_V)$ (this is single-side upper bound).
- 3) at most $\max\{\tau, |R_U|\} + \max\{\tau, |R_V|\} + j$ vertices in total, where j is the largest index such that $\sum_{i=1}^j \bar{d}_{w_i}(R_U \cup R_V) \leq k - \bar{e}(I) - \sum_{i=1}^{\tau - |R_U|} \bar{d}_{u_i}(R_V) - \sum_{i=1}^{\tau - |R_V|} \bar{d}_{v_i}(R_U)$.

Proof. We claim without proving that for any j , including j vertices of C_U to R_U introduces at least $\sum_{i=1}^j \bar{d}_{u_i}(R_V)$ more missing edges, where $(u_1, u_2, \dots)$ is an ordering of C_U in non-decreasing order regarding their non-degrees in R_V ; a similar claim can be made for including vertices of C_V to R_V .

For the single-side upper bound of item-1, any k -defective biclique in the instance I that includes j vertices of C_U to R_U will have at least $\bar{e}(I) + \sum_{i=1}^j \bar{d}_{u_i}(R_V) + \sum_{i=1}^{\tau - |R_U|} \bar{d}_{v_i}(R_U)$ missing edges (see Figure 4); the last term is necessary because the k -defective biclique must have at least τ vertices on the V -side. Thus, it is correct.

The upper bounds of item-2 and item-3 can be proved by following the same idea, and we omit the details. $\square$

We then consider upper bounds from the aspect of $R_U \cup R_V$.

Upper Bound (UB2). Any k -defective biclique in the instance $I = (g, R_U, R_V, C_U, C_V)$ contains

- at most $|R_U| + \min\{|C_U|, k - \bar{e}(I) + \min_{v \in R_V} d_v(C_U)\}$ vertices on the U -side (this is single-side upper bound).
- at most $|R_V| + \min\{|C_V|, k - \bar{e}(I) + \min_{u \in R_U} d_u(C_V)\}$ vertices on the V -side (this is single-side upper bound).
- at most $|R_U| + |R_V| + \min\{|C_U| + |C_V|, k - \bar{e}(I) + \min_{u \in R_U} d_u(C_V) + \min_{v \in R_V} d_v(C_U)\}$ vertices in total.

where $\min_{u \in R_U} d_u(C_V)$ is defined as $|C_V|$ if $R_U = \emptyset$ and $\min_{v \in R_V} d_v(C_U)$ is defined similarly.

Proof. The general idea of the proof is that for any $j \geq \min_{v \in R_V} d_v(C_U)$, including j vertices of C_U to R_U introduces at least $j - \min_{v \in R_V} d_v(C_U)$ more missing edges. The remaining of the proof is similar to that of **UB1** and is simpler. $\square$

From the proofs of **UB1** and **UB2**, we can see that the former is always tighter than the latter. Nevertheless, we still keep the latter because it can be computed more efficiently.

For reduction rules, we first prove the following one that greedily moves a vertex from $C_U \cup C_V$ to $R_U \cup R_V$.

Reduction Rule (RR3). Given an instance $I = (g, R_U, R_V, C_U, C_V)$ with $|R_V| \geq \tau$, for a vertex $u \in C_U$ satisfying $\bar{d}_u(R_U \cup C_V) = 1$ and $(R_U \cup u, R_V)$ being a k -defective biclique, we can greedily add u to R_U . Vertices of C_V can be handled similarly.

Proof. Let's consider a maximum k -defective biclique (R_U^*, R_V^*) in the instance I that does not contain u , where $|R_V^*| \geq \tau$, $\bar{d}_u(R_U^* \cup C_V) = 1$ and $(R_U^* \cup u, R_V^*)$ is a k -defective biclique. Let $v \in R_V^* \cup C_V$ be the non-neighbor of u . Then, v must be in R_V^* ; otherwise, $(R_U^* \cup u, R_V^* \cup v)$ would be a larger solution. If $v \in C_V$, $(R_U^* \cup u, R_V^* \setminus v)$ is also a maximum k -defective biclique by noting that $|R_V^* \setminus v| \geq |R_V^*| \geq \tau$. Otherwise $v \in R_V$. Then, there must exist either a vertex $u' \in R_U^* \setminus R_U$ satisfying $\bar{d}_{u'}(R_V^*) \geq 1$ or a vertex $v' \in R_V^* \setminus R_V$ satisfying $\bar{d}_{v'}(R_U^*) \geq 1$ (otherwise, $(R_U^* \cup u, R_V^*)$ would be a larger solution); in either case, we can remove this u' or v' from the solution (R_U^*, R_V^*) and add u to the solution. This completes the proof. $\square$

RR3 is an extension of **RR2** to handle vertices with exactly one non-neighbor. Note that, for **RR3** to be applied, certain conditions should be satisfied (e.g., $|R_V| \geq \tau$ or $|R_U| \geq \tau$).

The following reduction rule will utilize the size of the currently found best solution (R_U^*, R_V^*) for pruning. For presentation simplicity, we use lb to denote this size. Then, k -defective bicliques of size at most lb will be of no interest and can be safely excluded from consideration.

Reduction Rule (RR4). Given an instance $I = (g, R_U, R_V, C_U, C_V)$ and an edge (u, v) in g , if $d_u(R_V \cup C_V) + d_v(R_U \cup C_U) + k - \bar{e}(I) \leq lb$, then there is no k -defective biclique in I that contains both u and v and more than lb vertices in total.

Proof. Let's consider another instance $I' = (g, \{u\}, \{v\}, C'_U, C'_V)$ where $C'_U = R_U \cup C_U \setminus u$ and $C'_V = R_V \cup C_V \setminus v$. According to **UB2**, any k -defective biclique in I' has at most $1 + 1 + k - \bar{e}(I) + d_u(C'_V) + d_v(C'_U) \leq d_u(R_V \cup C_V) + d_v(R_U \cup C_U) + k - \bar{e}(I) \leq lb$ vertices in total. This completes the proof. $\square$

Suppose $d_u(R_V \cup C_V) + d_v(R_U \cup C_U) + k - \bar{e}(I) \leq lb$. The actual usage of **RR4** is as follows. If $u \in R_U$ and $v \in R_V$, then we prune the entire instance I . If $u \in C_U$ and $v \in C_V$, then we remove the edge (u, v) from g . Otherwise, we remove the vertex of $\{u, v\}$ that is in $C_U \cup C_V$ from the graph g (and also from the corresponding C_U or C_V).

Time complexity analysis. Firstly, it is easy to see that for the upper bounds, both **UB1** and **UB2** can be computed in time linear to the number of edges in g (i.e., $\mathcal{O}(m)$), by noting that sorting vertices in **UB1** can be conducted in linear time by counting sort. Secondly, for the reduction rules, **RR3** can be exhaustively applied until convergence (i.e., until the instance can no longer be reduced by it) in linear time, while **RR4** can be exhaustively applied until convergence in $\sum_{u \in R_U \cup C_U} (d_u(R_V \cup C_V))^2 + \sum_{v \in R_V \cup C_V} (d_v(R_U \cup C_U))^2 = \mathcal{O}(m\Delta)$ time. Note that, **RR4** is

Algorithm 3: Preprocess($G = (U, V, E), k, \tau$)

```

1 for each  $u \in U$  do
2    $S \leftarrow \emptyset$ ;
3   for each  $v \in N_u(V)$  do
4     for each  $u' \in N_v(U)$  do
5       if  $u' \notin S$  then  $S \leftarrow S \cup u'$ ;  $u'.cn \leftarrow 0$ ;
6        $u'.cn \leftarrow u'.cn + 1$ ;
7   for each  $v \in N_u(V)$  do
8      $deg \leftarrow 0$ ;
9     for each  $u' \in N_v(U) \setminus u$  do
10      if  $u'.cn \geq \tau - k$  then  $deg \leftarrow deg + 1$ ;
11      if  $1 + deg < \tau - k$  then Remove  $(u, v)$  from  $G$ ;
12 for each  $v \in V$  do
13   // Run a similar process to Lines 2-11

```

efficient in practice since its time complexity is also bounded by $\mathcal{O}\left(\sum_{(u,v) \text{ is removed}} (d_u(R_V \cup C_V) + d_v(R_U \cup C_U))\right)$ time. In addition, we also formulate reduction rules based on **UB1** and **UB2** to be conducted in linear time; that is, for a vertex $w \in C_U \cup C_V$, if the upper bound of the instance after including w into $R_U \cup R_V$ is no larger than lb , then we remove w from $C_U \cup C_V$. Note that, for time efficiency consideration, we do not compute the upper bound here as tight as **UB1** and **UB2**; we omit the details. Thus, P_1 in Theorem 4.2 is $\mathcal{O}(m\Delta)$.

Pruning in preprocessing. In addition to using the above reduction rules in Branch&Bound, we also propose to shrink the input graph G in a preprocessing (i.e., at the beginning of Algorithm 2) based on the size constraint τ and the size lb of the currently found best solution or a heuristically computed solution. For example, (1) **RR4** can be invoked on the instance $(\emptyset, \emptyset, U, V)$ to remove edges from G , and (2) all vertices of degree smaller than $\tau - k$ can be removed from G (following **UB2**). Furthermore, we prove the following lemma.

Lemma 5.1: For a k -defective biclique (R_U, R_V) that contains at least τ vertices on each side, any two vertices from the same side must have at least $\tau - k$ common neighbors.

Based on Lemma 5.1, we propose another preprocessing algorithm to remove edges from G . The general idea is as follows. For an arbitrary edge (u, v) , we first initialize $R_U = \{u\}$, $C_U = U \setminus u$, $R_V = \{v\}$, $C_V = V \setminus v$, and then remove from C_U all vertices that do not share at least $\tau - k$ common neighbors with u (following Lemma 5.1); let C'_U be the surviving vertices of C_U . If $1 + d_v(C'_U) < \tau - k$, then there is no k -defective biclique that contains both u and v and contains at least τ vertices on each side (following **UB1**), and thus we can remove the edge (u, v) from the graph G . The pseudocode of our preprocessing algorithm, denoted Preprocess, is given in Algorithm 3. Note that, instead of processing each edge (u, v) independently, we compute C'_U for all the adjacent edges of u (Lines 2–6), which is $\{u' \in S \mid u'.cn \geq \tau - k\}$. Lines 8–11 remove an edge (u, v) from G if $1 + d_v(C'_U) < \tau - k$. It is easy to see that the time complexity of Algorithm 3 is $\sum_{u \in U} (d_u(V))^2 + \sum_{v \in V} (d_v(U))^2 = \mathcal{O}(m\Delta)$.

B. Progressive Bounding

Both the single-side upper bounds (in **UB1** and **UB2**) and the preprocessing (in Algorithm 3) rely on the user-given size threshold τ , served as *single-side lower bound*, for pruning. However, this user-given size threshold τ typically will be significantly smaller than the actual sizes of R_U^* and R_V^* , denoted by τ_U^* and τ_V^* , in the optimal solution (R_U^*, R_V^*) , rendering the pruning to be less effective. Inspired by [12], we use the progressive bounding idea to manually set larger single-side lower bounds by enumerating possible ranges of τ_v^* . Specifically, let $\tau_v^{(0)} = |V| + 1$, and $\tau_v^{(t)} = \lfloor \tau_v^{(t-1)} / 2 \rfloor$ for $t \geq 1$. Then, there are $\mathcal{O}(\log |V|)$ possible ranges for τ_v^* , and the t -th range is $[\tau_v^{(t)}, \tau_v^{(t-1)})$ (i.e., left closed and right open) for $t \geq 1$. For the t -th range (i.e., iteration), the upper bound of τ_v^* is $\tau_v^{(t-1)} - 1$, and thus the lower bound of τ_u^* is $\tau_u^{(t)} = lb + 2 - \tau_v^{(t-1)}$, where lb is the size of the currently found best solution. Note that, we now have separate single-side lower bounds for the two sides (i.e., $\tau_u^{(t)}$ and $\tau_v^{(t)}$) and also a V -side upper bound (i.e., $\tau_v^{(t-1)} - 1$).

Algorithm 4: MDBC_{PB}($G = (U, V, E)$, k, τ)

```

1  $(R_U^*, R_V^*) \leftarrow (\emptyset, \emptyset)$ ;
2  $\tau_V^{(0)} \leftarrow |V| + 1$ ;  $t \leftarrow 0$ ;
3 while  $\tau_V^{(t)} > \tau$  do
4    $t \leftarrow t + 1$ ;
5    $\tau_V^{(t)} \leftarrow \max\{\tau, \lfloor \tau_V^{(t-1)} / 2 \rfloor\}$ ;
6    $\tau_U^{(t)} \leftarrow \max\{\tau, |R_U^*| + |R_V^*| + 2 - \tau_V^{(t-1)}\}$ ;
7    $(R_U^*, R_V^*) \leftarrow \text{MDBC}(G, k, \tau_U^{(t)}, \tau_V^{(t)}, \tau_V^{(t-1)}, R_U^*, R_V^*)$ ;
8 return  $(R_U^*, R_V^*)$ ;
```

The pseudocode of our algorithm equipped with progressive bounding, denoted MDBC_{PB}, is shown in Algorithm 4. For the t -th iteration, the size thresholds $\tau_u^{(t)}$ and $\tau_v^{(t)}$ are computed based on the above discussions (Lines 5–6). Then, we invoke MDBC with inputs $\tau_U^{(t)}, \tau_V^{(t)}, \tau_V^{(t-1)}, R_U^*, R_V^*$ to compute the largest k -defective biclique (R_U', R_V') (Line 7) such that $|R_U'| \geq \tau_u^{(t)}$, $\tau_v^{(t)} \leq |R_V'| < \tau_v^{(t-1)}$ and $|R_U'| + |R_V'| > |R_U^*| + |R_V^*|$. Note that, here MDBC slightly modifies Algorithm 2 by taking as input an initial (R_U^*, R_V^*) (and thus no need to initialize them at Line 1), and pruning a backtracking instance $I = (g, R_U, R_V, C_U, C_V)$ when $R_V \geq \tau_v^{(t-1)}$.

C. Iterative Computation

It is not hard to imagine that the search space for finding the maximum k -defective biclique grows dramatically when k increases; that is, the maximum s -defective biclique can be found much more efficiently than the maximum $(s + 1)$ -defective biclique for any $s \geq 0$. Moreover, the maximum $(s + 1)$ -defective biclique would not be significantly larger than the maximum s -defective biclique, as we prove in the lemma below.

Lemma 5.2: *For any $s \geq 0$ and a bipartite graph G with s -defective bicliques containing at least τ vertices on each side, the size of the maximum $(s + 1)$ -defective biclique with at*

least τ vertices on each side exceeds the size of the maximum s -defective biclique with the same property by at most 1.

Motivated by the above, we propose an iterative computation framework to iteratively compute the maximum s -defective biclique with at least τ vertices on each side, for s ranging from 0 to k . It finds the maximum k -defective biclique when the algorithm terminates. Suppose the algorithm has found a maximum s -defective biclique (R_U, R_V) , then for finding the maximum $(s + 1)$ -defective biclique, the algorithm can set a lower bound of $lb = |R_U| + |R_V|$ as well as an upper bound of $ub = |R_U| + |R_V| + 1$. This lower bound is so tight that it enhances the effectiveness of the pruning techniques presented in Section V-A. Also, this tight upper bound can be utilized to improve the efficiency from two aspects. Firstly, if a solution with size equal to ub is found, the algorithm can immediately return it as the maximum $(s + 1)$ -defective biclique. Secondly, for the t -th iteration of our progressive bounding algorithm MDBC_{PB}, we can also compute a U -side upper bound as $ub - \tau_v^{(t)}$ such that we can prune a backtracking instance once R_U is larger than this threshold.

Algorithm 5: MDBC_{PBIC}($G = (U, V, E)$, k, τ)

```

1  $(R_U^*, R_V^*) \leftarrow (\emptyset, \emptyset)$ ;
2  $lb \leftarrow 0$ ;  $ub \leftarrow |U| + |V|$ ;
3 for  $s \leftarrow 0$  to  $k$  do
4    $(R_U^*, R_V^*) \leftarrow \text{MDBC}_{\text{PB}}(G, s, \tau, R_U^*, R_V^*, ub)$ ;
5   if  $R_U^* \neq \emptyset$  then
6      $lb \leftarrow |R_U^*| + |R_V^*|$ ;  $ub \leftarrow |R_U^*| + |R_V^*| + 1$ ;
7 return  $(R_U^*, R_V^*)$ ;
```

Algorithm 5 illustrates the iterative computation algorithm, denoted MDBC_{PBIC}, which adds an additional layer to MDBC_{PB} for iteratively increasing the number of missing edges. Note that, MDBC_{PB} now also takes as input an initial (R_U^*, R_V^*) and a size upper bound ub .

Choose k . A silent feature of our iterative computation algorithm is that it enables us to not specify k in the input, but stop the algorithm as demand. That is, we can invoke Algorithm 5 without specifying k (or let it be ∞) to compute the maximum k -defective biclique for k starting from 0, and stop the execution when either the current result meets our requirement or it takes excessive time to get the next result.

D. Heuristically Compute a Large Solution in MDBC_{PB}

In this subsection, we discuss how to efficiently find a large k -defective biclique at Line 1 of Algorithm 4 (i.e., MDBC_{PB}), which is invoked by MDBC_{PBIC}. Recall that, MDBC_{PBIC} invokes Algorithm 4 by providing an initial (R_U, R_V) which may potentially be empty. There are two cases.

The first case is that the initial (R_U, R_V) provided to Algorithm 4 is not empty, which means that it is the optimal $(k - 1)$ -defective biclique. Then, it is also a k -defective biclique and we try to expand it into a larger k -defective biclique by fixing one side of the k -defective biclique as R_U or R_V . The pseudocode is shown in Algorithm 6, denoted Expand. We first

Algorithm 6: $\text{Expand}(G = (U, V, E), k, \tau, R_U, R_V)$

Input: A graph G , integers k and τ , and a k -defective biclique (R_U, R_V) s.t. $\min\{|R_U|, |R_V|\} \geq \tau$
Output: A large k -defective biclique with at least τ vertices on each side

- 1 $(v_1, v_2, \dots) \leftarrow$ an ordering of V based on $\bar{d}_v(R_U)$;
- 2 $R'_V \leftarrow$ the longest prefix of $(v_1, v_2, \dots)$ such that (R_U, R'_V) is a k -defective biclique;
- 3 $(u_1, u_2, \dots) \leftarrow$ an ordering of U based on $\bar{d}_u(R_V)$;
- 4 $R'_U \leftarrow$ the longest prefix of $(u_1, u_2, \dots)$ such that (R'_U, R_V) is a k -defective biclique;
- 5 **if** $|R_U| + |R'_V| > |R'_U| + |R_V|$ **then return** (R_U, R'_V) ;
- 6 **else return** (R'_U, R_V) ;

fix the U -side of the k -defective biclique to be R_U (Lines 1–2). It can be proved that the largest V -side that can form a k -defective biclique together with R_U is the longest prefix of the ordering of V , based on their non-degrees in R_U , such that the sum of their non-degrees is at most k ; let R'_V be such a prefix. Similarly, we fix the V -side to be R_V and greedily construct the best R'_U (Lines 3–4). Finally, we return the larger one among these two as the heuristic solution (Lines 5–6). Note that, both R'_V and R'_U exist and $\min\{|R'_V|, |R'_U|\} \geq \tau$. We observe that quite often, Algorithm 6 finds a larger solution than the initial (R_U, R_V) , which then is the optimal k -defective biclique, according to Lemma 5.2.

Algorithm 7: $\text{Heuristic}(G = (U, V, E), k, \tau)$

Output: A large k -defective biclique with at least τ vertices on each side

- 1 Compute a degeneracy ordering for the vertices of G ;
- 2 $(R_U, R_V) \leftarrow (\emptyset, \emptyset)$;
- 3 **while** $|R_U| < \tau$ **or** $|R_V| < \tau$ **do**
- 4 **if** $|R_U| \leq |R_V|$ **then**
- 5 $u \leftarrow \arg \min_{u' \in U \setminus R_U} \bar{d}_{u'}(R_V)$;
- 6 $R_U \leftarrow R_U \cup u$;
- 7 **else**
- 8 $v \leftarrow \arg \min_{v' \in V \setminus R_V} \bar{d}_{v'}(R_U)$;
- 9 $R_V \leftarrow R_V \cup v$;
- 10 **if** (R_U, R_V) is not a k -defective biclique **then return** $(\emptyset, \emptyset)$;
- 11 **else return** $\text{Expand}(G, k, \tau, R_U, R_V)$;

When the initial (R_U, R_V) provided to Algorithm 4 is empty, we construct a solution from scratch by first iteratively adding vertices to R_U and R_V until they both contain τ vertices and then invoking Algorithm 6 to expand it. The pseudocode is shown in Algorithm 7, denoted Heuristic. We first compute a degeneracy ordering for the vertices of G by treating it as a unipartite graph (Line 1), and initialize (R_U, R_V) as $(\emptyset, \emptyset)$ (Line 2). Then, we iteratively add into (R_U, R_V) a vertex with the fewest non-neighbors in $R_U \cup R_V$, by alternating between the two sides in a round-robin manner; ties are broken by preferring a vertex that appears later in the degeneracy ordering. We stop the process once both sides have τ vertices. If (R_U, R_V) has more than k missing edges, the algorithm fails to find a solution and returns $(\emptyset, \emptyset)$ (Line 10). Otherwise, it invokes Algorithm 6 to expand it to a larger solution.

TABLE II: Statistics of the datasets

Dataset	$ U $	$ V $	$ E $	$d_{\max}(U)$	$d_{\max}(V)$
YouTube	94,238	30,087	293,360	1,035	7,591
Actor	127,823	383,640	1,470,404	294	646
Cite	153,277	731,769	2,411,819	189,295	1,646
IMDB	303,617	896,302	3,782,463	1,334	1,590
Amazon	2,146,057	1,230,915	5,838,041	12,217	3,146
Wiki	18,038	2,192,849	6,561,379	916,505	30,606
DBLP	1,953,085	5,624,219	12,282,059	1,386	287
Google	5,998,790	4,443,631	20,592,962	423	95,165

TABLE III: Solution size for different k values

	Dataset	k					
		0	1	2	3	4	5
Small τ	Youtube	53	54	55	56	57	58
	Actor	51	52	53	54	54	54
	Cite	831	832	833	834	835	836
	IMDB	90	91	92	93	94	95
	Amazon	74	75	76	77	78	79
	Wiki	711	712	713	714	715	716
	DBLP	39	40	40	41	41	42
	Google	35	36	37	38	39	40
Large τ	Youtube	0	0	0	28	28	29
	Actor	0	0	18	19	19	20
	Cite	150	150	150	150	150	150
	IMDB	38	39	40	41	42	43
	Amazon	28	29	29	30	30	31
	Wiki	43	44	45	46	47	48
	DBLP	38	38	38	38	38	39
	Google	0	0	0	22	23	24

VI. EXPERIMENTS

In this section, we evaluate the effectiveness of the maximum k -defective biclique computation problem and the efficiency of our practical techniques. We implemented our algorithm in C++ and compiled it with the `-O3` optimization flag.³ All experiments are conducted in single-thread mode on a machine equipped with an Intel Core i9-11900K CPU and 64GB of main memory, and running Ubuntu 18.04. A timeout threshold of 3600 seconds is applied to each execution.

Datasets. We use 8 real-world bipartite graphs that are downloaded from <http://konect.cc/> in our experiments. Statistics of the datasets are shown in Table II, where the datasets are ordered in increasing order regarding the number of edges, $d_{\max}(U) = \max_{u \in U} d_u(V)$ and $d_{\max}(V) = \max_{v \in V} d_v(U)$.

Parameters. In our experiments, we vary the value of k from 0 to 5, and set $k = 4$ by default. For this default k , we select two τ values: a relatively small value of $\tau = 5$ and a relatively large value of $\tau = 14$. Two exceptions are the datasets *Actor* and *Google*, where no 4-defective biclique exists for $\tau = 14$. Therefore, we select τ values of 9 and 11 for these two datasets, respectively, as these are the largest τ values for which a valid 4-defective biclique can be found.

A. Effectiveness Testings

Exp-1: Impact of k on the solution size. In this experiment, we investigate how the maximum k -defective biclique size changes when k increases. The results are shown in Table III. We observe three patterns. First, in most cases, when k increases by 1, the solution size also increases by 1. This implies

³The source code can be found at <https://github.com/zhiyiwang1115/defectiveBiclique>.

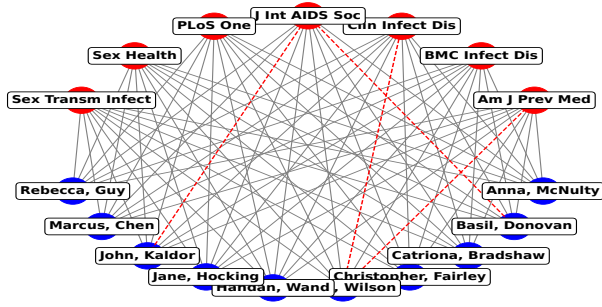


Fig. 5: Case study on MapAffil (red dashed lines: missing edges)

that in these cases, the maximum k -defective biclique is an extension of a maximum biclique by including k more vertices. Nevertheless, not all maximum bicliques can be extended to maximum defective bicliques, and thus it is infeasible to first find a single maximum biclique and then try to extend it. We remark that, although a single maximum k -defective biclique only contains up-to k more vertices than a maximum biclique, up-to $k \times r$ more vertices could be included in the result when aiming to find r solutions. Second, for datasets such as Youtube, Actor, and Google with large τ , there are no bicliques with at least τ vertices on each side. However, increasing k to 2 or 3 allows the algorithm to identify solutions, demonstrating the power of the defective biclique model in discovering meaningful patterns in challenging datasets. Third, for certain datasets such as Cite with $\tau = 14$, the size of the maximum defective biclique does not change when k increases from 0 to 5. This indicates that all the maximum bicliques are relatively stable and sparsely connected to other vertices.

Exp-2: Defective biclique vs biclique. Here, we evaluate the effectiveness of defective bicliques over bicliques by conducting a case study on the MapAffil dataset [43], which contains the information of author-publish-in-journal in PubMed. We filtered the dataset by considering only authors from Australia and publications in 2015. Figure 5 illustrates a maximum 4-defective biclique for $\tau = 7$. A maximum biclique for the same τ is a subgraph of the maximum 4-defective biclique by removing three researchers, David Wilson, John Kaldor, and Basil Donovan, who have published in most but not all of the journals in the group. Thus, the defective biclique model introduces robustness over biclique by allowing a small number of missing connections, thereby capturing a more comprehensive set of researchers with shared research interests.

Exp-3: Defective biclique vs biplex. We now evaluate various community structure metrics, including density, (bipartite) clustering coefficient [44], and average path length, for subgraphs extracted using the maximum k -defective biclique (with k ranging from 1 to 4) and the maximum k -biplex (with k ranging from 1 to 2). As the latter is much more time-consuming to compute, we choose a different set of smaller graphs in this experiment. The results are illustrated in Table IV, where $\tau = 5$. We can see that, increasing the parameter k relaxes the structural constraints of the extracted subgraphs.

TABLE IV: Defective biclique vs biplex

Dataset	Subgraph	Density	Clust. Coeff.	Avg. Path
SRWiki (402, 1823, 4784)	1-Defective	0.9917	0.9833	1.7094
	2-Defective	0.9840	0.9680	1.7218
	3-Defective	0.9769	0.9549	1.7333
	4-Defective	0.9704	0.9417	1.7440
	1-Biplex	0.9643	0.9286	1.7538
	2-Biplex	0.9394	0.8821	1.7937
UCForum (899, 522, 7089)	1-Defective	0.9667	0.9333	1.4909
	2-Defective	0.9429	0.8857	1.5303
	3-Defective	0.9250	0.8500	1.5641
	4-Defective	0.9111	0.8294	1.5934
	1-Biplex	0.9111	0.8222	1.5934
	2-Biplex	0.8667	0.7504	1.6765
Escorts (9456, 5841, 34796)	1-Defective	0.9750	0.9500	1.5128
	2-Defective	0.9556	0.9111	1.5495
	3-Defective	0.9400	0.8800	1.5809
	4-Defective	0.9273	0.8545	1.6083
	1-Biplex	0.9273	0.8545	1.6083
	2-Biplex	0.8750	0.7627	1.7143
DVD (16923, 14800, 65043)	1-Defective	0.9800	0.9600	1.5429
	2-Defective	0.9636	0.9323	1.5750
	3-Defective	0.9500	0.9128	1.6029
	4-Defective	0.9385	0.8880	1.6275
	1-Biplex	0.9286	0.8571	1.6491
	2-Biplex	0.8889	0.7898	1.7233

TABLE V: Solution shape when maximizing the number of edges vs. maximizing the number of vertices

	τ	Maximize edge count	Maximize vertex count
$k = 1$	2	(38, 2, 75)	(38, 2, 75)
	3	(3, 17, 50)	(3, 17, 50)
	4	(4, 10, 39)	(4, 10, 39)
	5	(5, 6, 29)	(5, 6, 29)
$k = 2$	3	(3, 18, 52)	(3, 18, 52)
	4	(4, 11, 42)	(4, 11, 42)
	5	(6, 6, 34)	(5, 7, 33)
	6	(6, 6, 34)	(6, 6, 34)

This relaxation is reflected in a decrease in both the density and clustering coefficient and an increase in average path length for both defective bicliques and bipoles. Additionally, we note that the biplex model is significantly more relaxed compared to the defective biclique model. In particular, for all 1-biplex cases, the identified communities are less dense than those in the 4-defective biclique model. Moreover, as k increases to 2, the biplex structure becomes even less compact, whereas the defective biclique model exhibits a gradual decrease in density, maintaining a more structured and cohesive community.

Exp-4: Maximize vertex count vs. maximize edge count. In our problem, we aim to maximize the number of vertices in the solution. As it is also natural to maximize the number of edges, we compare the solution shape of these two variants. To do this, we also implemented a prototype algorithm for the edge-variant of the problem, without advanced and tailed pruning techniques. As a result, the edge-variant algorithm can only handle small graphs, and we in this experiment use the *ucforum* dataset with $|U| = 899$, $|V| = 522$, and $|E| = 7089$. The solution shapes for different k and τ values are shown in Table V, where the first and second numbers represent the sizes of each side of the biclique, and the third number represents the total number of edges. We can see that in 7 out of 8 testings, the two variants return the same solution shape. The only exception is when $k = 2$ and $\tau = 5$, where the solution shape is also close to each other. Thus, we define our problem by maximizing the number of vertices, which is expected to be more efficiently solvable than the edge-variant.

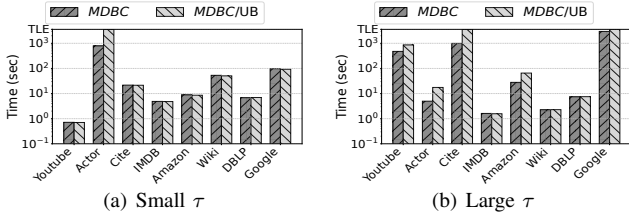


Fig. 6: Effect of upper-bound techniques

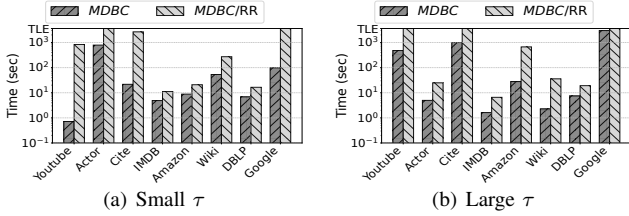


Fig. 7: Effect of reduction rules

B. Efficiency Testings

As none of the existing algorithms can be used to solve our problem, we in this subsection mainly test the effectiveness of our various practical techniques, introduced in Section V, in improving the efficiency. We remark that a baseline algorithm without any of these pruning technique will not be able to finish any of the testings within the time limit. Also note that, MDBC in this subsection actually refers to MDBC_{PBIC}.

Exp-5: Effect of upper-bound techniques. In this experiment, we evaluate the effectiveness of our proposed upper-bound techniques in improving the efficiency. Specifically, we compare MDBC with MDBC/UB, which is MDBC without upper-bound-based pruning. The results are shown in Figure 6. When τ is small, our upper-bound techniques show limited improvement, except for the dataset Actor. However, when τ is large, the upper-bound techniques significantly enhance performance on datasets such as YouTube, Actor, Cite, Amazon, and Google. These results are reasonable, as we have formulated exhaustive reduction rules based on **UB1**, which reduces the pruning opportunities of **UB1** in most cases. Consequently, the primary pruning power comes from **UB2**. Given that **UB2** considers filling both sides of the defective biclique to sizes approaching τ , it demonstrates greater pruning power when τ is large compared to when it is small. However, the computation of our upper-bound techniques remains efficient and does not introduce significant overhead, ensuring the overall effectiveness of our approach.

Exp-6: Effect of reduction rules. In this experiment, we evaluate the effectiveness of our proposed reduction techniques in improving the efficiency. Specifically, we compare MDBC with MDBC/RR, which is MDBC without reduction rules. The results are shown in Figure 7. We can see that our reduction rules provide general and unified pruning effectiveness across different graphs and parameter settings. This is intuitive: when τ is relatively small, MDBC can find a good solution quickly, and our reduction rules leverage the best-found solution to reduce vertices effectively. Conversely, when τ is relatively

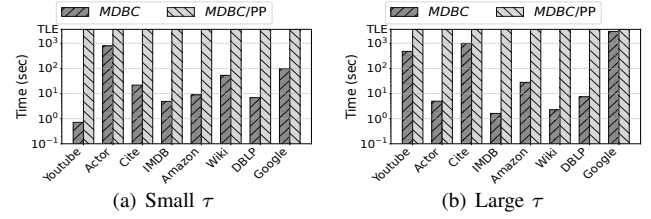


Fig. 8: Effect of preprocessing

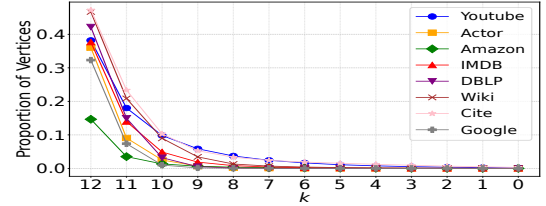


Fig. 9: Proportion of remaining vertices after preprocessing

large, finding a good solution becomes more challenging. However, our reduction rules also prune instances based on the value of τ . As a result, a significant difference between τ and k further enhances the efficiency of our pruning methods.

Exp-7: Effect of preprocessing. In this experiment, we evaluate the effectiveness of our preprocessing, as discussed at the end of Section V-A, in improving the efficiency by comparing MDBC with MDBC/PP which is MDBC without preprocessing. The results are shown in Figure 8. We can see that preprocessing significantly reduces the running time, due to eliminating a substantial portion of vertices. Figure 9 further reports the proportion of remaining vertices after applying preprocessing for $\tau = 14$ and varying k from 12 to 0. The results indicate that even when the difference $\tau - k$ is as small as 2, the preprocessing techniques can dramatically reduce the candidate set to less than 50%. For larger differences, the candidate set is further narrowed down, significantly improving the efficiency of MDBC by reducing the number of candidates to process. Our preprocessing step leverages the property that any vertex needs to have at least $\tau - k$ neighbors, and any two vertices on the same side must share at least $\tau - k$ common neighbors. As k decreases, $\tau - k$ increases, tightening the constraint on both neighbors and common neighbors. This results in a substantial reduction in the number of vertices satisfying the condition, effectively pruning a large portion of the candidate set and enhancing overall efficiency.

Exp-8: Effect of progressive bounding. In this experiment, we evaluate the effectiveness of progressive bounding by comparing MDBC with MDBC/PRG which is MDBC without progressive bounding. The results are shown in Figure 10. When τ is relatively small (*i.e.*, 5 in this case), MDBC consistently outperforms MDBC/PRG across all computations, with performance gains ranging from approximately two times (*e.g.*, IMDB) to over twenty times (*e.g.*, Wiki), as the progressive bounding framework can manually enlarge the size threshold τ . However, when τ is relatively large, progressive bounding demonstrates limited improvement. Nonetheless, it introduces minimal computational overhead in such cases.

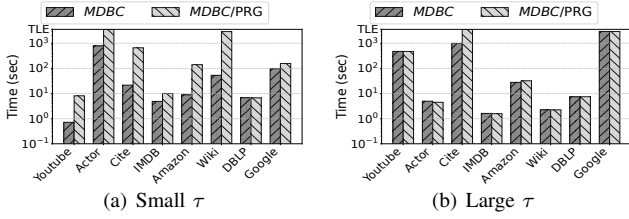


Fig. 10: Effect of progressive bounding

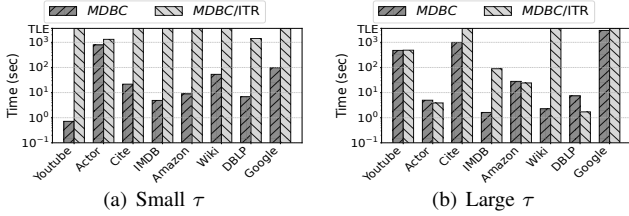


Fig. 11: Effect of iterative computation

Exp-9: Effect of iterative computation. In this experiment, we evaluate the effectiveness of the iterative computation framework by comparing MDBC with MDBC/ITR which is MDBC without iterative computation. The results are reported in Figure 11 which shows that iterative computation significantly improves the efficiency when τ is small. For instance, when $\tau = 5$, datasets such as Amazon, Cite, Google, IMDB, WIKI, and YouTube cannot be processed by MDBC/ITR within the one-hour time limit, while MDBC finishes within 100 seconds. This improvement aligns with our theoretical analysis in Section V-C. However, when τ becomes relatively large, iterative computation does not always yield significant improvements and may even lead to slow down in some cases. For example, for datasets such as Actor, Amazon, and DBLP, MDBC is slower than MDBC/ITR. This is because, for these graphs with large τ , iterative computation often fails to find an initial solution for s that is smaller than k , by noticing that many of these datasets do not contain valid 3-defective bicliques; note that, here $k = 4$. Nevertheless, the overhead is minor and acceptable compared to the benefits the framework provides for general k and τ settings.

Exp-10: Effect of heuristically finding an initial solution. In this experiment, we evaluate the effectiveness of heuristically finding an initial solution by comparing MDBC with MDBC/INIT which is MDBC without heuristically finding an initial solution. The results are reported in Figure 12 which shows that in almost all cases, MDBC performs at least as well as MDBC/INIT. When τ is relatively small, it becomes easier to find an initial solution using Heuristic. Moreover, it is more likely to identify a valid defective biclique with fewer than 4 missing edges, allowing Expand to effectively refine solutions during the iterative computation process. As a result, the effectiveness of initial solutions is significantly better when τ is set to a small value. However, when τ is relatively large, the possibility of finding an initial solution diminishes. In such cases, it becomes increasingly challenging to find a valid heuristic solution. For datasets such as Amazon,

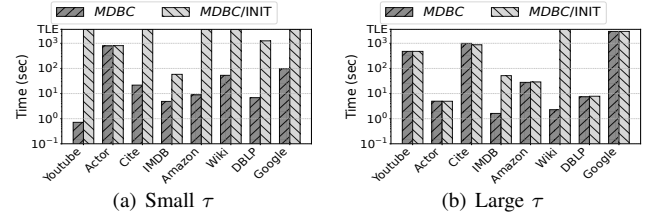


Fig. 12: Effect of heuristically finding an initial solution

TABLE VI: Running time of MDBC for different k values

	Dataset	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
$\tau - k = 1$	YouTube	0.95	1.08	0.99	0.71	0.71
	Actor	2.42	1.67	467.09	789.52	867.76
	Cite	37.55	27.14	32.32	21.70	12.96
	IMDB	9.77	7.15	5.91	4.88	4.42
	Amazon	20.43	13.45	11.15	8.85	8.27
	Wiki	-	3026.56	598.09	53.28	37.73
	DBLP	31.21	15.91	8.37	6.82	53.42
	Google	2212.47	936.52	256.25	97.34	53.28
$\tau - k = 5$	YouTube	0.62	0.70	1.12	0.97	0.75
	Actor	0.65	0.62	0.63	4.97	867.76
	Cite	12.21	28.34	12.35	7.17	-
	IMDB	3.37	6.80	2.70	2.60	2.57
	Amazon	5.44	4.95	4.87	5.03	5.22
	Wiki	27.14	8.03	6.38	5.71	5.02
	DBLP	3.16	4.71	6.70	8.20	9.23
	Google	45.13	24.64	15.25	12.78	-

Google, YouTube, Actor, DBLP, and Cite, Heuristic fails to find an initial solution when no 3-defective bicliques exist. Nevertheless, the computational overhead incurred in these cases is relatively minor.

Exp-11: Running time for different k values. In this experiment, we evaluate the efficiency of our algorithm MDBC for different k values by fixing $\tau - k$. We remark that, τ is not a fixed value here; otherwise, the running time will not decrease when k increases, due to our iterative computation framework. The results for $\tau - k = 1$ and $\tau - k = 5$ are reported in Table VI, which shows that the running time highly depends on the dataset. This is because for some graphs, extending an existing solution directly to an optimal one is relatively easy. For more challenging graphs, the bottleneck lies in determining whether a k -defective biclique exists with a size one larger than a $(k - 1)$ -defective biclique.

VII. CONCLUSION

In this paper, we introduced the problem of finding the maximum k -defective biclique in bipartite graphs, and developed general frameworks to solve it in $\mathcal{O}^*(\gamma^{n+k})$ and $\mathcal{O}^*(\gamma^{\alpha\Delta^2+k})$ time, where $\gamma \approx 1.8393$ is a constant. This also improves the state-of-the-art time complexity for maximum defective clique computation over traditional unipartite graphs. Furthermore, we proposed several practical techniques to improve the practical efficiency of our algorithm. Extensive empirical studies on 8 real bipartite graphs demonstrated the effectiveness and efficiency of our techniques.

Acknowledgements. This work was supported by the Australian Research Council Fundings of DP220103731, and the Research Grants Council of Hong Kong, China under No. 14205520.

REFERENCES

- [1] M. Ley, "The DBLP computer science bibliography: Evolution, research issues, perspectives," in *Proc. SPIRE 2002*, ser. Lecture Notes in Computer Science, A. H. F. Laender and A. L. Oliveira, Eds., vol. 2476. Springer, 2002, pp. 1–10. [Online]. Available: https://doi.org/10.1007/3-540-45735-6_1
- [2] J. Wang, A. P. de Vries, and M. J. T. Reinders, "Unifying user-based and item-based collaborative filtering approaches by similarity fusion," in *Proc. SIGIR, 2006*, E. N. Efthimiadis, S. T. Dumais, D. Hawking, and K. Järvelin, Eds. ACM, 2006, pp. 501–508. [Online]. Available: <https://doi.org/10.1145/1148170.1148257>
- [3] H. Li, J. Li, and L. Wong, "Discovering motif pairs at interaction sites from protein sequences on a proteome-wide scale," *Bioinform.*, vol. 22, no. 8, pp. 989–996, 2006. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btl020>
- [4] X. Wang and J. Liu, "A comparative study of the measures for evaluating community structure in bipartite networks," *Inf. Sci.*, vol. 448–449, pp. 249–262, 2018. [Online]. Available: <https://doi.org/10.1016/j.ins.2018.03.036>
- [5] Z. Zhang and Y. Ahn, "Community detection in bipartite networks using weighted symmetric binary matrix factorization," *CoRR*, vol. abs/1502.04428, 2015. [Online]. Available: <http://arxiv.org/abs/1502.04428>
- [6] Z. Chen, Y. Zhao, L. Yuan, X. Lin, and K. Wang, "Index-based biclique percolation communities search on bipartite graphs," in *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*. IEEE, 2023, pp. 2699–2712. [Online]. Available: <https://doi.org/10.1109/ICDE55515.2023.00207>
- [7] C. Maier and D. A. Simovici, "On biclique connectivity in bipartite graphs and recommendation systems," in *Proc. of ICISDM'21*. ACM, 2021, pp. 151–156.
- [8] T. Murata, "Discovery of user communities from web audience measurement data," in *Proc. IEEE/WIC/ACM International Conference on WI, 2004*. IEEE Computer Society, 2004, pp. 673–676. [Online]. Available: <https://doi.org/10.1109/WI.2004.10154>
- [9] K. Eren, M. Deveci, O. Küçüktunç, and Ü. V. Çatalyürek, "A comparative analysis of biclustering algorithms for gene expression data," *Briefings Bioinform.*, vol. 14, no. 3, pp. 279–292, 2013.
- [10] A. Prelic, S. Bleuler, P. Zimmermann, A. Wille, P. Bühlmann, W. Gruissem, L. Hennig, L. Thiele, and E. Zitzler, "A systematic comparison and evaluation of biclustering methods for gene expression data," *Bioinform.*, vol. 22, no. 9, pp. 1122–1129, 2006.
- [11] Q. Dai, R. Li, X. Ye, M. Liao, W. Zhang, and G. Wang, "Hereditary cohesive subgraphs enumeration on bipartite graphs: The power of pivot-based approaches," *Proc. ACM Manag. Data*, vol. 1, no. 2, pp. 138:1–138:26, 2023.
- [12] B. Lyu, L. Qin, X. Lin, Y. Zhang, Z. Qian, and J. Zhou, "Maximum and top-k diversified biclique search at scale," *VLDB J.*, vol. 31, no. 6, pp. 1365–1389, 2022.
- [13] J. Pattillo, N. Youssef, and S. Butenko, "On clique relaxation models in network analysis," *Eur. J. Oper. Res.*, vol. 226, no. 1, pp. 9–18, 2013.
- [14] H. Yu, A. Paccanaro, V. Trifonov, and M. Gerstein, "Predicting interactions in protein networks by completing defective cliques," *Bioinform.*, vol. 22, no. 7, pp. 823–829, 2006. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btl014>
- [15] R. Henriques and S. C. Madeira, "Bicnet: Flexible module discovery in large-scale biological networks using biclustering," *Algorithms Mol. Biol.*, vol. 11, p. 14, 2016. [Online]. Available: <https://doi.org/10.1186/s13015-016-0074-8>
- [16] X. Chen, Y. Zhou, J. Hao, and M. Xiao, "Computing maximum k -defective cliques in massive graphs," *Comput. Oper. Res.*, vol. 127, p. 105131, 2021. [Online]. Available: <https://doi.org/10.1016/j.cor.2020.105131>
- [17] L. Chang, "Efficient maximum k -defective clique computation with improved time complexity," *Proc. ACM Manag. Data*, vol. 1, no. 3, pp. 209:1–209:26, 2023. [Online]. Available: <https://doi.org/10.1145/3617313>
- [18] L. Chang, "Maximum defective clique computation: Improved time complexities and practical performance," *Proc. VLDB Endow.*, vol. 18, no. 2, pp. 200–212, 2024. [Online]. Available: <https://www.vldb.org/pvldb/vol18/p200-chang.pdf>
- [19] B. Liu, L. Yuan, X. Lin, L. Qin, W. Zhang, and J. Zhou, "Efficient (α, β) -core computation in bipartite graphs," *VLDB J.*, vol. 29, no. 5, pp. 1075–1099, 2020. [Online]. Available: <https://doi.org/10.1007/s00778-020-00606-9>
- [20] K. Wang, X. Lin, L. Qin, W. Zhang, and Y. Zhang, "Towards efficient solutions of bitruss decomposition for large-scale bipartite graphs," *VLDB J.*, vol. 31, no. 2, pp. 203–226, 2022. [Online]. Available: <https://doi.org/10.1007/s00778-021-00658-5>
- [21] K. Yu, C. Long, S. Liu, and D. Yan, "Efficient algorithms for maximal k -biplex enumeration," in *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022*, Z. G. Ives, A. Bonifati, and A. E. Abbadi, Eds. ACM, 2022, pp. 860–873. [Online]. Available: <https://doi.org/10.1145/3514221.3517847>
- [22] X. Liu, J. Li, and L. Wang, "Quasi-bicliques: Complexity and binding pairs," in *Computing and Combinatorics, 14th Annual International Conference, COCOON 2008, Dalian, China, June 27-29, 2008, Proceedings*, ser. Lecture Notes in Computer Science, X. Hu and J. Wang, Eds., vol. 5092. Springer, 2008, pp. 255–264. [Online]. Available: https://doi.org/10.1007/978-3-540-69733-6_26
- [23] A. Abidi, R. Zhou, L. Chen, and C. Liu, "Pivot-based maximal biclique enumeration," in *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020, C. Bessiere, Ed. ijcai.org, 2020*, pp. 3558–3564. [Online]. Available: <https://doi.org/10.24963/ijcai.2020/492>
- [24] L. Chen, C. Liu, R. Zhou, J. Xu, and J. Li, "Efficient maximal biclique enumeration for large sparse bipartite graphs," *Proc. VLDB Endow.*, vol. 15, no. 8, pp. 1559–1571, 2022. [Online]. Available: <https://www.vldb.org/pvldb/vol15/p1559-chen.pdf>
- [25] J. Chen, K. Wang, R. Li, H. Qin, X. Lin, and G. Wang, "Maximal biclique enumeration: A prefix tree based approach," in *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*. IEEE, 2024, pp. 2544–2556. [Online]. Available: <https://doi.org/10.1109/ICDE60146.2024.00200>
- [26] J. Yang, J. Yang, Z. Ma, C. Zhang, S. Yang, and W. Zhang, "Efficient maximal biclique enumeration on large uncertain bipartite graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12634–12648, 2023. [Online]. Available: <https://doi.org/10.1109/TKDE.2023.3272110>
- [27] J. Yang, Y. Peng, D. Ouyang, W. Zhang, X. Lin, and X. Zhao, "(p,q)-biclique counting and enumeration for large sparse bipartite graphs," *VLDB J.*, vol. 32, no. 5, pp. 1137–1161, 2023. [Online]. Available: <https://doi.org/10.1007/s00778-023-00786-0>
- [28] R. Peeters, "The maximum edge biclique problem is np-complete," *Discret. Appl. Math.*, vol. 131, no. 3, pp. 651–654, 2003. [Online]. Available: [https://doi.org/10.1016/S0166-218X\(03\)00333-0](https://doi.org/10.1016/S0166-218X(03)00333-0)
- [29] S. Shahinpour, S. Shirvani, Z. Ertem, and S. Butenko, "Scale reduction techniques for computing maximum induced bicliques," *Algorithms*, vol. 10, no. 4, p. 113, 2017. [Online]. Available: <https://doi.org/10.3390/a10040113>
- [30] E. Shaham, H. Yu, and X. Li, "On finding the maximum edge biclique in a bipartite graph: a subspace clustering approach," in *Proc. SIAM, 2016*, S. C. Venkatasubramanian and W. M. Jr., Eds. SIAM, 2016, pp. 315–323. [Online]. Available: <https://doi.org/10.1137/1.9781611974348.36>
- [31] Q. Feng, S. Li, Z. Zhou, and J. Wang, "Parameterized algorithms for edge biclique and related problems," *Theor. Comput. Sci.*, vol. 734, pp. 105–118, 2018. [Online]. Available: <https://doi.org/10.1016/j.tcs.2017.09.027>
- [32] C. McCreesh and P. Prosser, "An exact branch and bound algorithm with symmetry breaking for the maximum balanced induced biclique problem," in *International Conference on AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems, 2014*, ser. Lecture Notes in Computer Science, H. Simonis, Ed., vol. 8451. Springer, 2014, pp. 226–234. [Online]. Available: https://doi.org/10.1007/978-3-319-07046-9_16
- [33] Y. Zhou, A. Rossi, and J. Hao, "Towards effective exact methods for the maximum balanced biclique problem in bipartite graphs," *Eur. J. Oper. Res.*, vol. 269, no. 3, pp. 834–843, 2018. [Online]. Available: <https://doi.org/10.1016/j.ejor.2018.03.010>
- [34] L. Chen, C. Liu, R. Zhou, J. Xu, and J. Li, "Efficient exact algorithms for maximum balanced biclique search in bipartite graphs," in *Proc. of SIGMOD'21*, 2021, pp. 248–260.
- [35] K. Wang, W. Zhang, X. Lin, L. Qin, and A. Zhou, "Efficient personalized maximum biclique search," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 2022, pp. 498–511. [Online]. Available: <https://doi.org/10.1109/ICDE53745.2022.00042>

- [36] Y. Zhao, Z. Chen, C. Chen, X. Wang, X. Lin, and W. Zhang, "Finding the maximum k -balanced biclique on weighted bipartite graphs (extended abstract)," in *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*. IEEE, 2024, pp. 5697–5698. [Online]. Available: <https://doi.org/10.1109/ICDE60146.2024.00480>
- [37] J. Wang, J. Yang, C. Zhang, and X. Lin, "Efficient maximum edge-weighted biclique search on large bipartite graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 7921–7934, 2023. [Online]. Available: <https://doi.org/10.1109/TKDE.2022.3220901>
- [38] Q. Dai, R. Li, D. Cui, M. Liao, Y. Qiu, and G. Wang, "Efficient maximal biplex enumerations with improved worst-case time guarantee," *Proc. ACM Manag. Data*, vol. 2, no. 3, p. 135, 2024. [Online]. Available: <https://doi.org/10.1145/3654938>
- [39] K. Yu and C. Long, "Maximum k -biplex search on bipartite graphs: A symmetric- k branching approach," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 49:1–49:26, 2023. [Online]. Available: <https://doi.org/10.1145/3588729>
- [40] W. Luo, K. Li, X. Zhou, Y. Gao, and K. Li, "Maximum biplex search over bipartite graphs," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*. IEEE, 2022, pp. 898–910. [Online]. Available: <https://doi.org/10.1109/ICDE53745.2022.00072>
- [41] J. Gao, Z. Xu, R. Li, and M. Yin, "An exact algorithm with new upper bounds for the maximum k -defective clique problem in massive sparse graphs," in *Proc. AAAI, 2022*. AAAI Press, 2022, pp. 10 174–10 183. [Online]. Available: <https://doi.org/10.1609/aaai.v36i9.21257>
- [42] M. Jin, J. Zheng, and K. He, "Kd-club: An efficient exact algorithm with new coloring-based upper bound for the maximum k -defective clique problem," in *Proc. AAAI, 2024*, M. J. Wooldridge, J. G. Dy, and S. Natarajan, Eds. AAAI Press, 2024, pp. 20 735–20 742. [Online]. Available: <https://doi.org/10.1609/aaai.v38i18.30061>
- [43] V. I. Torvik, "Mapaffil 2016 dataset – pubmed author affiliations mapped to cities and their geocodes worldwide," 2018. [Online]. Available: https://doi.org/10.13012/B2IDB-4354331_V1
- [44] M. Latapy, C. Magnien, and N. D. Vecchio, "Basic notions for the analysis of large two-mode networks," *Soc. Networks*, vol. 30, no. 1, pp. 31–48, 2008. [Online]. Available: <https://doi.org/10.1016/j.socnet.2007.04.006>